Actividad de Aprendizaje

Semana 07

Indicaciones Generales:

1. El plagio se sanciona con la suspensión o expulsión del estudiante de la Universidad. Reglamento General de Estudios.
2. Sea cuidadoso con su redacción, la cual formará parte de su calificación (aspectos léxicos, sintácticos, semánticos).
3. Lea bien la pregunta o enunciado antes de proceder a su desarrollo, administre su tiempo eficazmente.

Ejercicios de Lambdas y Streams

Contenido

- Lambdas
- Streams
- Funciones de Streams

Programación Funcional

La programación funcional es un tipo de programación declarativa, basada en el uso de funciones, dotándolas de mayor versatilidad. Java, desde su versión 8, permite la programación funcional empleando funciones Lambda (a través de las Interfaces Funcionales) y el API Stream.

API STREAM:

A través del API Stream podemos trabajar sobre colecciones de una manera limpia y clara, evitando bucles y algoritmos que ralentizan los programas y hacen complejo entender su funcionalidad.

Existen 3 partes que componen un Stream que de manera general serían:

1.- Un Stream funciona a partir de una lista o colección, que también se la conoce como la fuente de donde obtienen información. El método *stream()* convertirá la colección en *Stream* para comenzar su tratamiento.

2.- Operaciones intermedias que actúan sobre el Stream. Estas son solo algunas:

- map: Obtiene un nuevo *Stream* resultado de aplicarle una función a cada elemento (en general una expresión Lambda que implementa la interfaz funcional *Function*).
- filter: selecciona elementos según la expresión pasada como parámetro que será una implementación de la interfaz funcional *Predicate*, generalmente mediante una expresión Lambda.
- sorted: ordena el *Stream*. Lo hace por el criterio por defecto de la clase (Comparable) o con el comparador pasado como parámetro, generalmente como expresión Lambda.
- skip *(n)*: elimina de *Stream* los 'n' primeros de elementos del *Stream*.
- limit *(n)*: se queda solo con los 'n' primeros elementos del *Stream*, eliminando los sobrantes.

3.- Operaciones terminales. Son la última operación que se hace con el *Stream*. Ejemplos:

- collect: Obtienen una colección con el resultado de los procesos previos aplicados al Stream.
- forEach: Itera sobre cada elemento de Stream (el método peek() valdría para iterar sobre el Stream como operación intermedia, no terminal).
- reduce: Permite realizar un cálculo sobre los elementos del Stream, utilizando un operador binario para definir la operación. Tiene muchas posibilidades, por ejemplo:
  - `.reduce(0, (a,b) -> a+b)` acumula, sería como a+=b, empezando con a=0.
  - `.reduce(Integer::sum)` hace lo mismo que la anterior, pero devuelve un *Optional*. Esta es una clase que almacena un valor, gestionando los valores nulos. Para obtener su valor hay que ejecutar su método *get* (luego lo veremos en los ejemplos)
  - `.reduce(Integer::min)` Análogo al anterior, pero con el mínimo.
  - `.reduce(Integer::max)` Análogo al anterior, pero con el máximo.
- count: Obtiene un *long* con la cantidad de elementos del Stream una vez procesado.
- sum: Obtiene la suma de los valores, previamente debemos hacer un *mapToInt (Integer::intValue)*

EJERCICIOS – GUIADO 01

- Crear un Proyecto en Netbeans con sus **apellidos del estudiante.**
- Agregar un paquete llamado **ejercicio01.**
- Agregar una Clase Main llamada **PruebaStream.java**

Para entender todas estas operaciones lo mejor es ver ejemplos en funcionamiento. Partiremos en todos los casos de una lista con los 10 primeros enteros.

```
List <Integer> numeros = Arrays.asList(1,2,3,4,5,6,7,8,9,10);
```

Ejemplo 1: Mostrar los elementos del ArrayList inicial elevados al cuadrado.

```
numeros.stream()
        .map(x->x*x)
        .forEach(y->System.out.println(y));
```

O también:

```
numeros.stream()
        .map(x->x*x)
        .forEach(System.out::println);
```

A veces, se presenta todo en una sola línea, pero se entiende peor:

```
numeros.stream().map(x->x*x).forEach(System.out::println);
```

Ejemplo 2: Obtener una nueva lista con elementos de la lista inicial elevados al cuadrado.

```
List cuadrados = numeros.stream()
                        .map(x->x*x)
                        .collect(Collectors.toList());
```

Ejemplo 3: Obtener un Set con elementos de la lista inicial que sean pares elevados al cuadrado:

```
Set cuadradosPares = numeros.stream()
                        .filter(x -> x%2==0)
                        .map(x->x*x)
                        .collect(Collectors.toSet());
```

Ejemplo 4: Mostrar el cuadrado de los elementos de las posiciones 4 a 8, ambas incluidas (empezando en cero).

```
numeros.stream()
        .skip(4)
        .limit(5)
        .map(x->x*x)
        .forEach(System.out::println);
```

Ejemplo 5: Obtener la suma de los elementos impares:

Este ejercicio podemos hacerlo de varias formas:

```
int res1 = numeros.stream()
            .filter(x -> x%2!=0)
            .reduce(0, (a, b) -> a + b);
System.out.println(res1);
```

O también:

```
int res2 = lista.stream()
            .map(z -> z.getMeGusta())
            .mapToInt(Integer::intValue)
            .sum();
System.out.println(res2);
```

Y también con esta última versión:

```
Optional res3 = lista.stream()
            .map(z -> z.getMeGusta())
            .reduce(Integer::sum);
System.out.println(res3.get());
```

Ejemplo 6: A partir de un ArrayList de String, hacer una lista con los que empiecen por "A":

```
List <String> textos = Arrays.asList("Alfa","Bravo","Charlie","Aback");
textos.stream()
        .filter(s->s.startsWith("A"))
        .forEach(System.out::println);
```

Ejemplo 7: A partir de una lista de "Peli" mostrar los títulos de las películas posteriores al año 1998, ordenadas alfabéticamente.

```
lista.stream()
     .filter(x -> x.getAño() > 1998)
     .map(x -> x.getNombre())
     .sorted()
     .forEach(System.out::println);
```

Ejemplo 8: A partir de una lista de "Peli" mostrar el título de la película más antigua:

```
Optional<Peli> var =lista.stream()
                .min( (a,b)->a.getAño()-b.getAño());
System.out.println(var.get().getNombre());
```

EJERCICIO – GUIADO 02

- Agregar al mismo Proyecto en netbeans un paquete con el nombre **ejercicio02**
- Agregar una Clase Main llamada **PruebaStreamOperaciones.java**

PruebaStreamOperaciones.java

```
package ejercicio02;

import java.util.*;
import java.util.stream.Collectors;
import java.util.stream.Stream;

/**
 * @author FREDDY
 */
public class PruebaStreamOperaciones
{
    public static void main(String[] args)
    {
        //Creación de una Colecciòn del tipo ArrayList
        List<String> lista = new ArrayList<>();
        //Se agrega nombres a la coleccion
        lista.add("Freddy");
        lista.add("Henrry");
        lista.add("Ana");
        lista.add("Briseida");
        lista.add("Carolina");
        lista.add("Lenin");
        lista.add("Eduardo");
        lista.add("Amparo");

        /*Stream.forEach ()
          forEach(), método que ayuda a iterar sobre todos los elementos de una secuencia
          y realizar alguna operación en cada uno de ellos. La operación a realizar se
          pasa como expresión lambda.
        */
        lista.forEach(System.out::println);
        lista.forEach(p -> System.out.println(p));
        //Usando forEach() en un Stream
        Stream<String> stream = lista.stream();
        stream.forEach(p -> System.out.println(p));
```

```java
/*
Stream.filter()
filter(), es un método que acepta un predicado para filtrar todos los elementos
del flujo. Esta operación es intermedia, lo que nos permite llamar a otra operación
de flujo (por ejemplo, forEach()) en el resultado.
*/
lista.stream().filter((s) -> s.startsWith("A"))
              .forEach(System.out::println);

/*
Stream.map ()
map(),es una operación intermedia que convierte cada elemento de la secuencia en
otro objeto a través de la función dada.
El sgte ejemplo convierte cada cadena en una cadena MAYÚSCULAS.
Pero también podemos usar map() para transformar un objeto en otro tipo.
*/
lista.stream().filter((s) -> s.startsWith("A"))
              .map(String::toUpperCase)
              .forEach(System.out::println);

/*
Stream.sorted ()
sorted(),es una operación intermedia que devuelve una vista ordenada de la secuencia.
Los elementos de la secuencia se ordenan en orden natural a menos que pasemos un
Comparador personalizado.
*/
lista.stream().sorted()
              .map(String::toUpperCase)
              .forEach(System.out::println);


/*Stream.collect ()
collect(), método que se utiliza para recibir elementos de un flujo(stream) y
almacenarlos en una colección.
*/
List<String> resMay = lista.stream().sorted()
                           .map(String::toUpperCase)
                           .collect(Collectors.toList());

System.out.print(resMay);
```

```
        /*Stream.match ()
        Se pueden utilizar varias operaciones de coincidencia para comprobar si un predicado
        determinado coincide con los elementos del flujo. Todas estas operaciones coincidentes
        son terminales y devuelven un boolean resultado.*/
        boolean res = lista.stream().anyMatch((s) -> s.startsWith("A"));

        System.out.println(res);

        res = lista.stream().allMatch((s) -> s.startsWith("A"));

        System.out.println(res);

        res = lista.stream().noneMatch((s) -> s.startsWith("A"));

        System.out.println(res);

        /*
        Stream.count ()
        count(), es una operación terminal que devuelve el número de elementos en la corriente
        como un long.
        */
        long total = lista.stream()
                          .filter((s) -> s.startsWith("A"))
                          .count();
        System.out.println(total);

        /*Stream.reduce()
        reduce(), método que realiza una reducción de los elementos del flujo con la función
        dada. El resultado se da en una variable de una clase Optional con el valor reducido.
        En el ejemplo dado, estamos reduciendo todas las cadenas concatenándolas usando un
        separador #.
        */

        Optional<String> reduced = lista.stream()
                                        .reduce((s1,s2) -> s1 + "#" + s2);
        reduced.ifPresent(System.out::println);
    }
}
```

EJERCICIO – GUIADO 03

- Agregar al mismo Proyecto en netbeans un paquete con el nombre **ejercicio03**
- Agregar una clase Main **PruebaStreamPares.java**

PruebaStreamPares.java

```
package ejercicio03;

import java.util.*;
import java.util.stream.*;
/**
 * @author FREDDY
 */
public class PruebaStreamPares {

    /*
    Recopilar elementos de Stream en una lista
    En el ejemplo dado, 1ª creamos una secuencia en números enteros del 1 al 10.
    Luego, procesamos los elementos de la secuencia para encontrar todos los números pares.
    Finalmente se reune todos los números pares en un List.
    */
    public static void main(String[] args) {
        List<Integer> lista = new ArrayList<Integer>();

        for(int i = 1; i<= 10; i++){
            lista.add(i);
        }

        Stream<Integer> stream = lista.stream();
        List<Integer> listaPares = stream.filter(i -> i%2 == 0)
                                         .collect(Collectors.toList());
        System.out.print(listaPares);

        long nro= lista.stream().filter((p) -> p>5).peek(System.out::println).count();
        System.out.println(nro);
    }
}
```

TAREA01 (Sobre el ejercicio03):

1. Mostrar la lista de números impares.
2. Mostrar la cantidad de números pares e impares.

TAREA02:

Agregar al mismo Proyecto en netbeans un **paquete** con el nombre **ejercicio04**

1. Agregar una Clase Main llamada “PruebaStreamCadenas.java”
2. Crear una colección de Cursos del área de Ciencias (al menos 8 cursos)
3. Mostrar todos los cursos.
4. Mostrar todos los cursos que inician con la letra “M”.
5. Mostrar en forma ordenada los cursos.
6. Mostrar la cantidad de cursos

Entregables:

- Prepara un documento basado en los puntos anteriores para exponerlos en clase.



Fecha límite de presentación: Determinados por el docente.
Integrantes: Determinados por el docente.